OP2 Assignment 2 Semaphores

May, 2026

Mihnea Bastea

Introduction

This project implements a supermarket checkout simulation in C using threads and semaphores. The program simulates customers, cashiers, and self checkout terminals running at the same time. Shared resources such as carts, scanners, and queues are synchronized using semaphores.

Customers arrive at random intervals, shop for a random amount of time, and then choose either a cashier or a self checkout terminal depending on whether they managed to get a scanner.

An additional feature was also implemented. Customers that use scanners have a 25% chance of being selected for a random check after using the self checkout terminal. This adds additional time to their total shopping time.

Implementation

General design

The supermarket was implemented as a concurrent system where multiple activities happen at the same time. Customers, cashiers, terminals, and the display are all executed using separate threads.

Each customer follows the same sequence of actions. A customer arrives, tries to take a cart, optionally takes a scanner, shops for a random amount of time, joins a queue, waits to be served, returns the cart, and then leaves the supermarket.

Customers that do not get a scanner join a cashier queue. Customers that use scanners join a shared terminal queue.

The following diagram shows the general customer flow inside the supermarket.

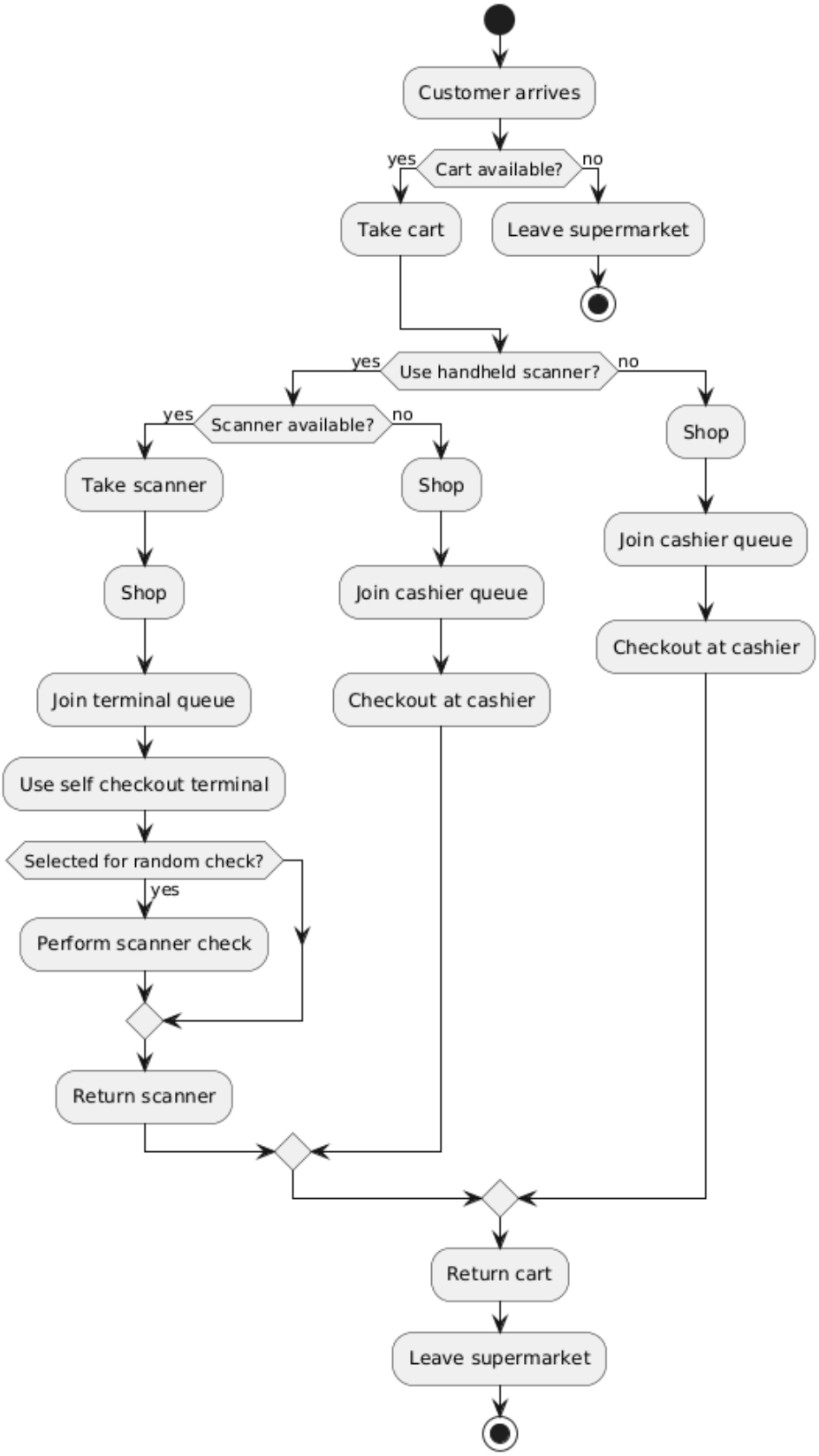


Figure 1: General customer flow.

The simulation uses semaphores to synchronize shared resources and queues. Limited resources such as carts and scanners are controlled using counting semaphores. Binary semaphores are used as mutexes for protecting shared queues and customer state.

Threads and synchronization

The simulation uses multiple types of threads that run independently at the same time. Customer threads simulate customers moving through the supermarket. Cashier threads process customers waiting at checkout lanes. Terminal threads process customers that use handheld scanners. A separate display thread updates the terminal output once per second and shows the current state of the simulation.

Shared resources such as carts, scanners, queues, and customer state are synchronized using semaphores. Counting semaphores are used for limited resources like carts and scanners, while binary semaphores are used for protecting shared data structures.

The cashier and terminal threads follow a producer consumer model. Customer threads act as producers by adding customers to queues, while cashier and terminal threads act as consumers by removing and processing customers from those queues.

```c
for (;;) {
    sem_wait(&cashier_customer_available[cashier_id]);

    sem_wait(&cashier_queue_mutex[cashier_id]);
    Customer *customer = queue_pop(&cashier_queues[cashier_id]);
    sem_post(&cashier_queue_mutex[cashier_id]);

    if (customer == NULL) {
        break;
    }

    set_customer_state(customer, STATE_AT_CASHIER);
    sleep_seconds(random_between(CASHIER_TIME_MIN, CASHIER_TIME_MAX));

    sem_post(&customer->done);
}
```

The `cashier_customer_available` semaphore is used to signal that a customer was added to the queue. This allows cashier threads to sleep while no customers are available instead of continuously checking the queue in a loop.

The queue itself is protected using `cashier_queue_mutex`. This prevents multiple threads from modifying the queue at the same time.

Each customer also owns a `done` semaphore. After joining a queue, the customer waits on this semaphore until checkout is complete. The cashier or terminal thread posts the semaphore after processing the customer, allowing the customer thread to continue.

The following sequence diagrams show the interaction between customer threads and worker threads for both checkout paths.

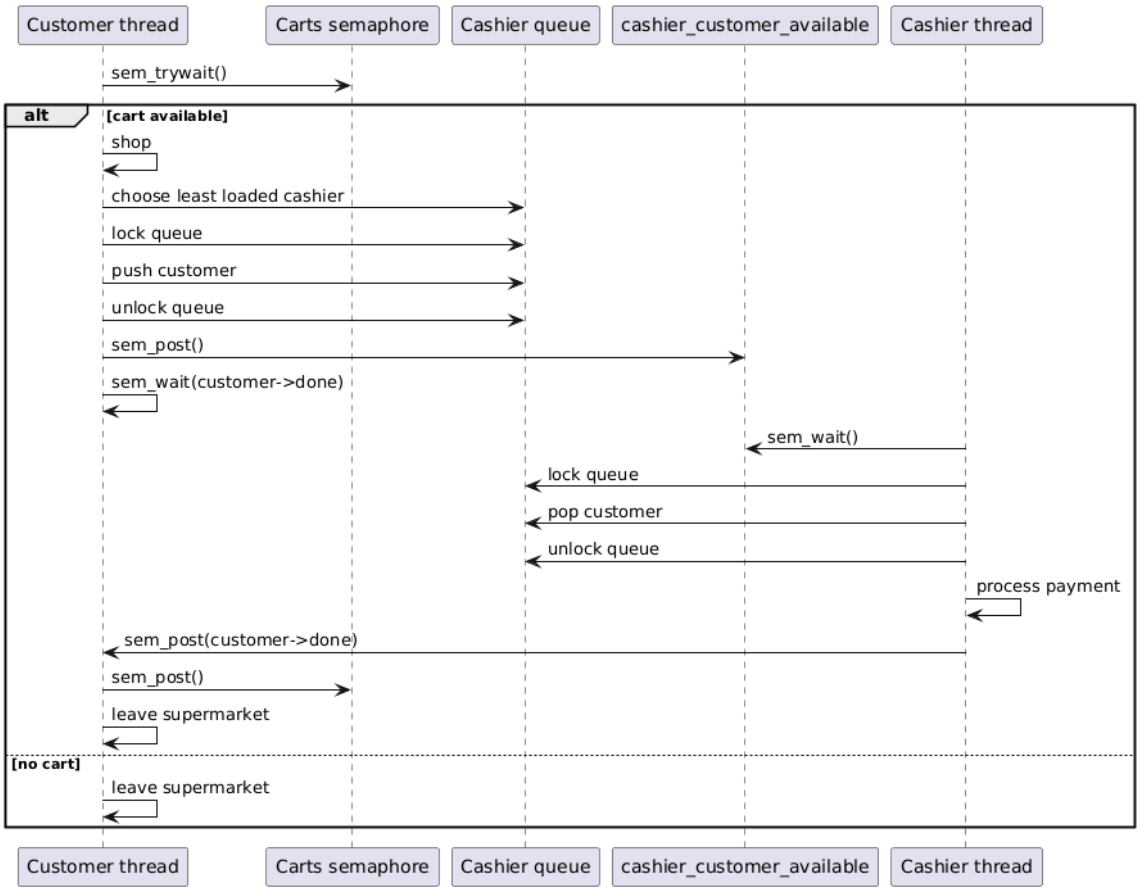


Figure 2: Sequence diagram for a customer using a cashier.

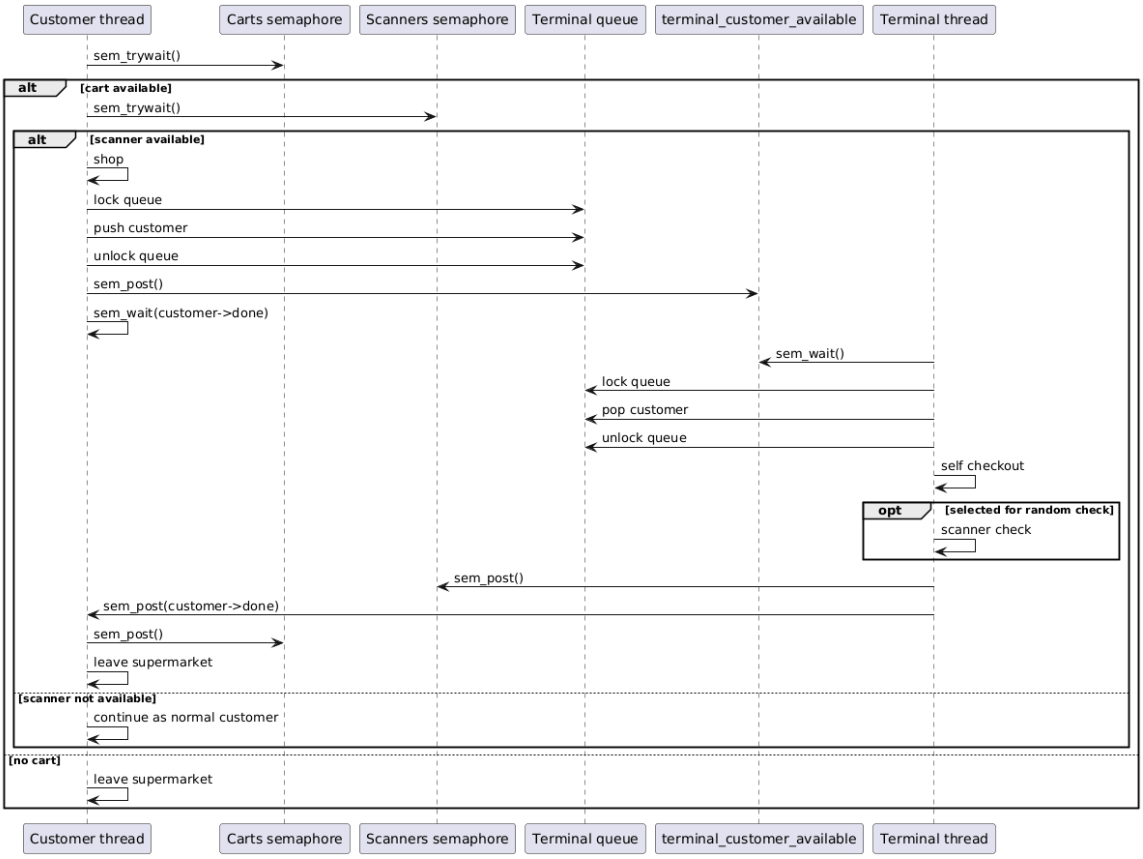


Figure 3: Sequence diagram for a customer using a handheld scanner.

Worker thread shutdown is implemented using the poison pill pattern. A `NULL` customer pointer is inserted into each queue when the supermarket closes. When a worker thread receives a `NULL` value from the queue, it exits its processing loop and terminates gracefully.

Queue management

Each cashier has its own queue implemented as a linked list. Customers that do not use handheld scanners always join the cashier with the smallest load.

The cashier load is calculated using both the number of waiting customers and whether the cashier is currently serving another customer. This prevents customers from joining a cashier that appears to have an empty queue while still serving someone.

```c
static int get_cashier_load(int cashier_id) {
    sem_wait(&cashier_queue_mutex[cashier_id]);
    int load = queue_count(&cashier_queues[cashier_id]);
    sem_post(&cashier_queue_mutex[cashier_id]);

    sem_wait(&customers_mutex);
    if (cashier_serving[cashier_id] != NULL) {
        load++;
    }
    sem_post(&customers_mutex);

    return load;
}
```

The customer then joins the cashier with the smallest calculated load.

```c
static int find_shortest_cashier_queue(void) {
    int selected_cashier = 0;
    int min_load = get_cashier_load(0);

    for (int i = 1; i < NUM_CASHIERS; i++) {
        int load = get_cashier_load(i);

        if (load < min_load) {
            min_load = load;
            selected_cashier = i;
        }
    }

    return selected_cashier;
}
```

Customers that use handheld scanners are handled differently. Instead of using separate queues for each terminal, all scanner customers join one shared terminal queue. The first available terminal processes the next customer from this queue.

Additional feature

The implemented additional feature is the random scanner check. This only applies to customers that use a handheld scanner.

After a scanner customer finishes the normal self checkout step, there is a 25% chance that the customer is selected for an extra check. If this happens, the customer enters the checking state and spends extra time at the terminal.

```c
set_customer_state(customer, STATE_AT_TERMINAL);
sleep_seconds(TERMINAL_TIME);

if (random_between(1, 4) == 1) {
    set_customer_state(customer, STATE_CHECKING);
    sleep_seconds(random_between(TERMINAL_CHECKING_TIME_MIN,
    TERMINAL_CHECKING_TIME_MAX));
}
```

The checking time is stored separately from the normal terminal time. This makes it visible in the final customer summary shown by the display.

Testing

The program was tested by changing the constants in `config.h`. This made it possible to force specific situations and check if the synchronization still worked correctly.

The screenshots were taken from the terminal display. The display shows each customer state, the cashier queues, the terminal queue, and the available carts and scanners.

Test 1: Normal run

The first test used the default configuration. In this run, both checkout paths are visible. Some customers used scanners and finished through the terminal queue, while others used the cashier queues.

The final overview also shows that all carts and scanners were returned. This means the resource semaphores were released correctly after customers finished.

```
Supermarket Checkout Simulator
Time: 42s

Customers:
C01 [S] | done         total:  9s (shopping:7s, waiting:0s, cashier:0s, terminal:1s, checking:0s, returning:1s)
C02 [S] | done         total:  7s (shopping:5s, waiting:0s, cashier:0s, terminal:1s, checking:0s, returning:1s)
C03 [C] | done         total: 11s (shopping:6s, waiting:0s, cashier:4s, terminal:0s, checking:0s, returning:1s)
C04 [C] | done         total: 15s (shopping:10s, waiting:0s, cashier:4s, terminal:0s, checking:0s, returning:1s)
C05 [S] | done         total: 12s (shopping:9s, waiting:0s, cashier:0s, terminal:1s, checking:0s, returning:2s)
C06 [S] | done         total: 21s (shopping:9s, waiting:0s, cashier:0s, terminal:1s, checking:10s, returning:1s)
C07 [C] | done         total: 16s (shopping:4s, waiting:0s, cashier:10s, terminal:0s, checking:0s, returning:2s)
C08 [S] | done         total: 21s (shopping:9s, waiting:0s, cashier:0s, terminal:1s, checking:9s, returning:2s)
C09 [C] | done         total: 15s (shopping:6s, waiting:0s, cashier:8s, terminal:0s, checking:0s, returning:1s)
C10 [C] | done         total: 17s (shopping:7s, waiting:0s, cashier:9s, terminal:0s, checking:0s, returning:1s)

Cashiers:
0: free        | queue:
1: free        | queue:
2: free        | queue:

Terminals:
0: free
1: free
Terminal queue:

Resources:
Carts:    20/20
Scanners: 10/10
```

Figure 4: Normal simulation run.

Test 2: No carts available

For this test, the number of carts was set to zero.

```c
#define NUM_CARTS 0
```

Since no carts were available, every customer left immediately. No customer entered the shopping state, and the display shows `left no cart` for all customers. This confirms that customers do not wait for carts when none are available.

```
Supermarket Checkout Simulator
Time: 29s

Customers:
C01 [-] | left         no cart
C02 [-] | left         no cart
C03 [-] | left         no cart
C04 [-] | left         no cart
C05 [-] | left         no cart
C06 [-] | left         no cart
C07 [-] | left         no cart
C08 [-] | left         no cart
C09 [-] | left         no cart
C10 [-] | left         no cart

Cashiers:
0: free        | queue:
1: free        | queue:
2: free        | queue:

Terminals:
0: free
1: free
Terminal queue:

Resources:
Carts:     0/0
Scanners: 10/10
```

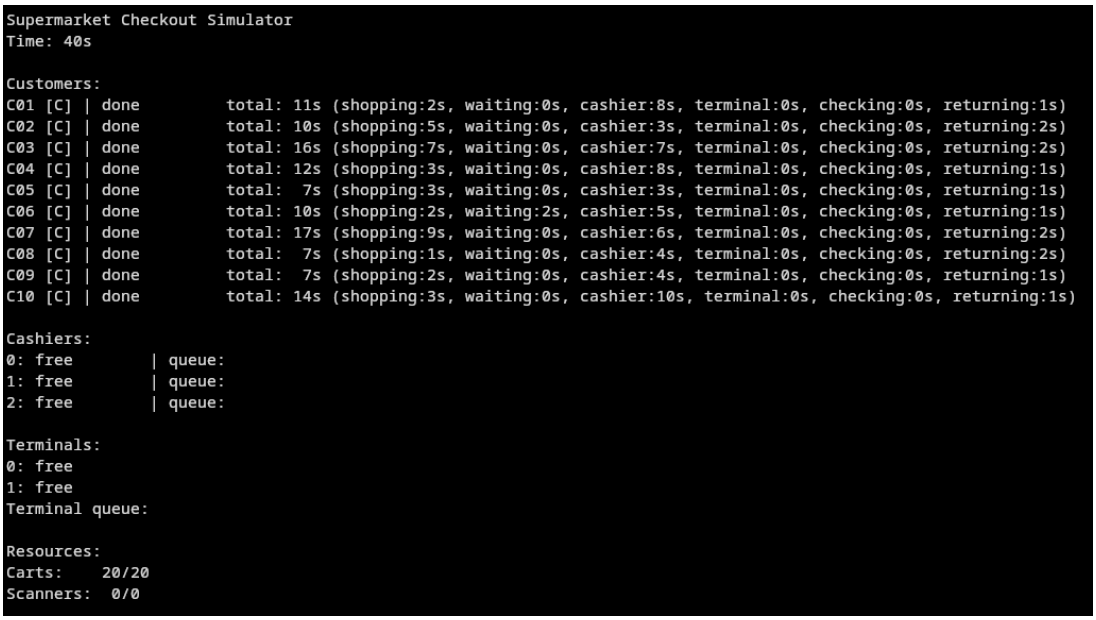
Figure 5: Customers leaving because no carts are available.

Test 3: No scanners available

For this test, the number of scanners was set to zero.

```c
#define NUM_SCANNERS 0
```

In this run, all customers used the cashier path. The terminal queue stayed empty and all finished customers are marked with `[C]`, which means cashier checkout. This confirms that customers continue normally when no scanner is available.



```
Supermarket Checkout Simulator
Time: 40s

Customers:
C01 [C] | done         total: 11s (shopping:2s, waiting:0s, cashier:8s, terminal:0s, checking:0s, returning:1s)
C02 [C] | done         total: 10s (shopping:5s, waiting:0s, cashier:3s, terminal:0s, checking:0s, returning:2s)
C03 [C] | done         total: 16s (shopping:7s, waiting:0s, cashier:7s, terminal:0s, checking:0s, returning:2s)
C04 [C] | done         total: 12s (shopping:3s, waiting:0s, cashier:8s, terminal:0s, checking:0s, returning:1s)
C05 [C] | done         total:  7s (shopping:3s, waiting:0s, cashier:3s, terminal:0s, checking:0s, returning:1s)
C06 [C] | done         total: 10s (shopping:2s, waiting:2s, cashier:5s, terminal:0s, checking:0s, returning:1s)
C07 [C] | done         total: 17s (shopping:9s, waiting:0s, cashier:6s, terminal:0s, checking:0s, returning:2s)
C08 [C] | done         total:  7s (shopping:1s, waiting:0s, cashier:4s, terminal:0s, checking:0s, returning:2s)
C09 [C] | done         total:  7s (shopping:2s, waiting:0s, cashier:4s, terminal:0s, checking:0s, returning:1s)
C10 [C] | done         total: 14s (shopping:3s, waiting:0s, cashier:10s, terminal:0s, checking:0s, returning:1s)

Cashiers:
0: free        | queue:
1: free        | queue:
2: free        | queue:

Terminals:
0: free
1: free
Terminal queue:

Resources:
Carts:    20/20
Scanners:  0/0
```

Figure 6: Simulation with no handheld scanners available.

Test 4: Slow cashiers

For this test, cashier processing was made slower.

```c
#define CASHIER_TIME_MIN 8
#define CASHIER_TIME_MAX 10
```

This made cashier checkout take longer, so customers spent more time in cashier processing. The screenshot shows cashier times between 8 and 10 seconds. This confirms that the configured timing values affect the simulation as expected.

This test is also useful for checking cashier load balancing, because slower cashiers make queues more likely to appear.

```
Supermarket Checkout Simulator
Time: 45s

Customers:
C01 [S] | done         total: 11s (shopping:9s, waiting:0s, cashier:0s, terminal:1s, checking:0s, returning:1s)
C02 [C] | done         total: 12s (shopping:2s, waiting:0s, cashier:9s, terminal:0s, checking:0s, returning:1s)
C03 [S] | done         total: 20s (shopping:10s, waiting:0s, cashier:0s, terminal:1s, checking:7s, returning:2s)
C04 [S] | done         total: 16s (shopping:8s, waiting:0s, cashier:0s, terminal:1s, checking:5s, returning:2s)
C05 [C] | done         total: 12s (shopping:1s, waiting:0s, cashier:10s, terminal:0s, checking:0s, returning:1s)
C06 [C] | done         total: 22s (shopping:10s, waiting:0s, cashier:10s, terminal:0s, checking:0s, returning:2s)
C07 [C] | done         total: 21s (shopping:10s, waiting:0s, cashier:10s, terminal:0s, checking:0s, returning:1s)
C08 [C] | done         total: 14s (shopping:3s, waiting:0s, cashier:9s, terminal:0s, checking:0s, returning:2s)
C09 [S] | done         total: 11s (shopping:9s, waiting:0s, cashier:0s, terminal:1s, checking:0s, returning:1s)
C10 [S] | done         total: 18s (shopping:10s, waiting:0s, cashier:0s, terminal:1s, checking:6s, returning:1s)

Cashiers:
0: free        | queue:
1: free        | queue:
2: free        | queue:

Terminals:
0: free
1: free
Terminal queue:

Resources:
Carts:    20/20
Scanners: 10/10
```

Figure 7: Simulation with slower cashier processing.

Conclusion

The simulation successfully implements the supermarket checkout system using threads and semaphores. Customers can take carts, optionally use scanners, shop, queue for checkout, pay, return their carts, and leave the supermarket.

The tests show that the main requirements work correctly. Customers leave when no cart is available, customers fall back to cashier checkout when no scanner is available, and cashier timing changes affect the checkout process.

The program also implements one additional feature. Scanner customers have a 25% chance of being selected for a random check, which adds extra time to their checkout process.

Overall, the simulation shows how semaphores can be used to control limited resources, protect shared queues, and synchronize customer threads with cashier and terminal worker threads.

Reflection

This project helped me better understand synchronization between threads and the practical use of semaphores. One of the more difficult parts was designing the queue system correctly and making sure that shared data could not be accessed by multiple threads at the same time.

Another important part was handling thread shutdown cleanly. The final version uses the poison pill pattern, which simplified the worker thread logic and removed the need for additional shutdown checks inside the processing loops.

The terminal display was also useful during development because it made it easier to observe customer behavior, queue states, and resource usage while the simulation was running.